

Composable AI Agent Stack

Architecture Guide

Agent-S + Browser Use + OpenHands unified through LiteLLM

Any API Key	Local or Cloud	Zero Vendor Lock-in	Self-Hosted
-------------	----------------	---------------------	-------------

Version 1.0 | May 2026

Table of Contents

1. Architecture Overview	3
1.1 The Four-Layer Architecture	3
1.2 How the Pieces Connect	3
2. LiteLLM: The LLM Gateway	4
2.1 Supported Providers	4
2.2 Key Features	4
2.3 Configuration Example	5
3. Agent Tool Integration	6
3.1 Agent-S: Full Desktop Control	6
3.2 Browser Use: Browser Automation	6
3.3 OpenHands: Software Development Agent	7
4. Deployment Guide	8
4.1 Quick Start (5 Minutes)	8
4.2 Local-Only Mode (No Cloud API Keys)	8
4.3 Cloud Deployment	8
5. Comparison: Composable Stack vs Open Computer Use	9
6. Use Case Matrix	9
7. Why This Stack Is Future-Proof	10

1. Architecture Overview

The Composable AI Agent Stack is a vendor-agnostic architecture that combines three best-of-breed open-source agent platforms with a unified LLM gateway. Instead of relying on a single monolithic platform like Open Computer Use (Coasty), which has severe vendor lock-in (Amazon Bedrock only, missing core executor, single contributor), this stack assembles independently swappable components that each excel at a specific type of computer automation. The result is a system that works with any API key, runs locally or in the cloud, and has zero vendor lock-in at every layer.

1.1 The Four-Layer Architecture

The stack is organized into four distinct layers, each independently configurable and replaceable. This separation of concerns means that swapping your LLM provider does not require changing any agent tool configuration, and replacing one agent tool does not affect the others. The layers communicate through standard OpenAI-compatible HTTP APIs, making the entire stack transparent and debuggable.

Layer	Component	Role	Replaceable With
Layer 2	Agent-S	Full desktop control (mouse, keyboard, screen)	CUA Driver, UFO
Layer 2	Browser Use	Browser automation (Playwright + AI)	Stagehand, Skyvern
Layer 2	OpenHands	Software development agent	Aider, SWE-agent
Layer 1	LiteLLM	Unified LLM gateway (OpenAI-compatible)	OpenRouter, vLLM
Layer 0	Ollama	Local model runner (optional)	vLLM, LM Studio, LocalAI

Table 1: Architecture layers and replaceable alternatives

1.2 How the Pieces Connect

All three agent tools connect to LiteLLM through its OpenAI-compatible API endpoint. LiteLLM acts as a transparent proxy: it receives requests in OpenAI format, routes them to the configured provider (OpenAI, Anthropic, Google, Ollama, etc.), and returns responses in the same format. The agent tools never see the actual provider and never need direct API keys. This architecture provides several critical advantages: centralized key management, automatic failover

between providers, unified cost tracking, load balancing across multiple deployments, and the ability to swap providers without touching any agent configuration.

Agent Tool	Connection Method	Config Parameter
Agent-S	OpenAI SDK (raw)	base_url=http://litellm:4000/v1
Browser Use	ChatOpenAI (LangChain-style)	base_url=http://litellm:4000/v1
OpenHands	LiteLLM SDK (native)	base_url=http://litellm:4000

Table 2: How each agent tool connects to LiteLLM

2. LiteLLM: The LLM Gateway

LiteLLM is the keystone of the entire architecture. It is an open-source AI gateway that provides a single OpenAI-compatible API to 140+ LLM providers and 2,600+ models. By placing LiteLLM between your agent tools and your LLM providers, you gain centralized key management, automatic failover, load balancing, cost tracking, and virtual key management with per-team budgets and rate limits. The proxy runs as a Docker container with a PostgreSQL database for persistent configuration, and includes a web dashboard for monitoring spend, managing keys, and viewing analytics.

2.1 Supported Providers

Category	Providers	Key Required?
Cloud APIs	OpenAI, Anthropic, Google Gemini, DeepSeek	Yes (per-provider)
Cloud Platforms	AWS Bedrock, Azure OpenAI, Google Vertex AI	Yes (cloud credentials)
Routers	OpenRouter, Together AI, Fireworks AI, Groq	Yes (router key)
Self-Hosted	Ollama, vLLM, LM Studio, LocalAI	No (runs locally)
OpenAI-Compatible	Any endpoint speaking OpenAI API format	Depends on endpoint

Table 3: LiteLLM provider categories

2.2 Key Features

- ⚖️ **Load Balancing:** Multiple deployments with the same model name are automatically load balanced. Routing strategies include least-busy, latency-based, and cost-based routing.

- â **Automatic Failover:** If a primary model fails, LiteLLM automatically tries configured fallback models in order. For example, if GPT-4o is unavailable, it can fall back to Claude Sonnet, then Gemini Flash.
- â **Virtual Keys:** Create API keys scoped to specific models, budgets, and rate limits. Issue separate keys per team or per agent tool with spend caps and expiration dates.
- â **Cost Tracking:** Automatic spend tracking per key, per user, per team, and per model. Built-in cost map for all known models with support for custom pricing.
- â **Self-Hosted:** Full control over your data and configuration. No information leaves your infrastructure. Deploy via Docker Compose with PostgreSQL for persistence.

2.3 Configuration Example

The following YAML configuration demonstrates a production-ready LiteLLM setup with multiple cloud providers, automatic failover, and load balancing. All API keys are read from environment variables, never hardcoded in the configuration file.

```
model_list:
- model_name: gpt-4o

litellm_params:
model: openai/gpt-4o
api_key: os.environ/OPENAI_API_KEY
- model_name: claude-sonnet

litellm_params:
model: anthropic/claude-sonnet-4-20250514
api_key: os.environ/ANTHROPIC_API_KEY
- model_name: llama3-local

litellm_params:
model: ollama/llama3
api_base: http://ollama:11434
router_settings:
routing_strategy: least-busy

fallbacks:
- model_name: gpt-4o

fallbacks: [claude-sonnet, llama3-local]

general_settings:
```

```
master_key: os.environ/LITELLM_MASTER_KEY
```

3. Agent Tool Integration

3.1 Agent-S: Full Desktop Control

Agent-S is the only open-source agent that provides true full-desktop control, making it the closest alternative to Coasty's computer use capabilities. It can move your mouse, type on your keyboard, take screenshots, and interact with any native desktop application including Excel, Photoshop, VS Code, and legacy enterprise software. Agent-S was the first agent to beat humans on the OSWorld benchmark (72.6% accuracy), demonstrating its effectiveness at general computer interaction tasks. The framework supports multiple LLM providers through its engine abstraction layer, and connecting it to LiteLLM requires zero code changes, just configuration of the `base_url` parameter.

LiteLLM Integration (zero code changes):

```
# CLI approach

python -m gui_agents.s3.cli_app \
--provider openai \
--model gpt-4o \
--model_url http://litellm:4000/v1 \
--model_api_key sk-litellm-key

# Programmatic approach

engine_params = {
    "engine_type": "openai",
    "model": "gpt-4o",
    "base_url": "http://litellm:4000/v1",
    "api_key": "sk-litellm-key",
}
```

3.2 Browser Use: Browser Automation

Browser Use is the most popular open-source browser automation agent with over 94,000 GitHub stars. It combines AI reasoning with Playwright browser automation to perform complex web tasks including form filling, web scraping, multi-tab navigation, and SaaS workflow automation. The framework supports 15+ LLM providers out of the box and has a built-in ChatLiteLLM wrapper class for direct LiteLLM integration. Browser Use also offers a desktop application and

cloud deployment options, making it suitable for both individual developers and enterprise deployments. Its MIT license ensures full commercial use without restrictions.

LiteLLM Integration (three methods, zero code changes):

```
# Method 1: ChatOpenAI with base_url (simplest)

from browser_use.llm.openai.chat import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o",
base_url="http://litellm:4000/v1",
api_key="sk-litellm-key")

# Method 2: ChatLiteLLM (full routing)

from browser_use.llm.litellm.chat import ChatLiteLLM

llm = ChatLiteLLM(model="openai/gpt-4o",
api_base="http://litellm:4000",
api_key="sk-litellm-key")
```

3.3 OpenHands: Software Development Agent

OpenHands is a full-featured AI software development agent with over 74,000 GitHub stars. It can write code, run tests, debug issues, create pull requests, and deploy applications, all within sandboxed execution environments. Unlike the other two tools, OpenHands uses LiteLLM as its native LLM SDK, meaning the integration is first-class and fully supported out of the box. OpenHands provides a web UI for interactive development, a CLI for automation, and an SDK for programmatic integration. Its MIT license and massive contributor community make it the most mature and well-supported option for software development automation.

LiteLLM Integration (first-class, zero code changes):

```
# Environment variables (used in docker-compose.yml)

LLM_MODEL=gpt-4o
LLM_API_KEY=sk-litellm-key
LLM_BASE_URL=http://litellm:4000

# Or via config.toml:

[llm]
model = "gpt-4o"
api_key = "sk-litellm-key"
base_url = "http://litellm:4000"
```

```
# Or via SDK:

from openhands.sdk.llm import LLM

llm = LLM(model="openai/gpt-4o",
api_key=SecretStr("sk-litellm-key"),
base_url="http://litellm:4000")
```

4. Deployment Guide

4.1 Quick Start (5 Minutes)

- â─ Step 1: Clone the stack configuration: `git clone <stack-repo> && cd composable-agent-stack`
- â─ Step 2: Configure API keys: `cp litellm/.env.example litellm/.env` then edit with your provider keys
- â─ Step 3: Start LiteLLM and Browser Use: `docker compose up -d litellm litellm-db browser-use`
- â─ Step 4: Verify LiteLLM is running: `curl http://localhost:4000/health/liveliness`
- â─ Step 5: Start OpenHands (optional): `docker compose up -d openhands`

4.2 Local-Only Mode (No Cloud API Keys)

For fully offline operation, start Ollama alongside the stack. LiteLLM will route requests to local models when no cloud providers are configured or when cloud providers are unavailable. This mode is ideal for air-gapped environments, sensitive data processing, or cost-free experimentation. Simply start the stack with the local-llm profile: `docker compose --profile local-llm up -d`. Then pull a model: `docker exec ollama ollama pull llama3`. LiteLLM will automatically detect and route to Ollama based on the model name configured in `config.yaml`.

4.3 Cloud Deployment

The same Docker Compose configuration works in cloud environments (AWS ECS, Google Cloud Run, Azure Container Instances) with minimal modifications. The key changes are: (1) replace localhost URLs with internal service DNS names, (2) add TLS termination via a load balancer or reverse proxy, (3) configure managed database services instead of the Docker PostgreSQL container for production reliability, and (4) set up proper authentication and network security groups. The architecture is cloud-agnostic by design; no cloud-specific services are required.

5. Comparison: Composable Stack vs Open Computer Use

The following table provides a direct comparison between the Composable AI Agent Stack and Open Computer Use (Coasty). The composable stack addresses every critical weakness identified in our deep audit of the Coasty platform, including the missing core executor, Bedrock-only LLM provider, single contributor risk, and lack of CI/CD infrastructure.

Criteria	Open Computer Use (Coasty)	Composable Stack
Full source available?	No (core executor missing)	Yes (all three tools fully open)
LLM provider support	Amazon Bedrock only	140+ providers via LiteLLM
Local LLM (Ollama)	Disabled in production	Fully supported
Custom API endpoint	Not supported	Any OpenAI-compatible endpoint
Desktop control	Yes (VM-based)	Yes (Agent-S, local or VM)
Browser automation	Yes (in-VM Selenium)	Yes (Browser Use + Playwright)
Software development	No dedicated agent	Yes (OpenHands sandbox)
License	Apache-2.0 (incomplete)	MIT + Apache-2.0 (complete)
Commercial use	Yes (but incomplete)	Yes (fully functional)
Self-hostable	Partial (needs Python backend)	Fully self-hosted
Contributors	1 person	100+ across three projects
CI/CD	None	Active CI in all three projects
Community size	~600 stars	180,000+ combined stars
Vendor lock-in risk	Severe (Bedrock, Supabase, Stripe)	None (every layer swappable)

Table 4: Detailed comparison between Open Computer Use and the Composable Stack

6. Use Case Matrix

Each agent tool in the composable stack excels at different types of automation tasks. The following matrix helps you select the right tool for each use case. For workflows that span multiple domains (desktop, browser, and code), the tools can

be composed together, with each handling its area of specialty while sharing the same LLM backend through LiteLLM.

Use Case	Best Tool	Why
Automate desktop apps (Excel, QuickBooks)	Agent-S	Only tool with full desktop control
Web scraping and form filling	Browser Use	94K+ stars, Playwright-based, most mature
Code generation and bug fixing	OpenHands	Sandboxed execution, self-correcting
RPA replacement	Agent-S + Browser Use	Desktop apps + web forms coverage
QA testing (web)	Browser Use	Cross-browser, automated screenshots
QA testing (desktop)	Agent-S	Native app interaction and validation
CI/CD automation	OpenHands	Git operations, test running, deployment
Lead generation	Browser Use	LinkedIn/Sales Navigator automation
Legacy system integration	Agent-S	Bridge old desktop apps to modern APIs
E-commerce management	Browser Use	Multi-platform product listing sync
Security auditing	OpenHands	Code scanning, vulnerability detection
Invoice processing	Agent-S + Browser Use	Desktop PDF + web portal submission

Table 5: Use case to tool mapping

7. Why This Stack Is Future-Proof

- â **No vendor lock-in:** Each layer is independently swappable. Replace LiteLLM with OpenRouter. Replace Browser Use with Stagehand. Replace OpenAI with Anthropic. Zero rewrite required because every component communicates through standard OpenAI-compatible APIs.
- â **Any API key, any endpoint:** Cloud keys (OpenAI, Anthropic, Google) or local models (Ollama, vLLM) or custom endpoints (LiteLLM proxy, corporate gateways). LiteLLM normalizes everything behind one endpoint. Your agent tools never see the difference.

â■ Local or cloud hosted: Run everything on your laptop with Ollama for zero-cost operation, or deploy to AWS/GCP/Azure with managed APIs for production scale. Same configuration, different .env file. The Docker Compose setup works identically in both environments.

â■ Community-backed: The three agent tools have a combined 180,000+ GitHub stars and hundreds of active contributors. If one project goes stale, alternatives exist at every layer. No single point of failure like the single-contributor Coasty project.

â■ Standards-based: All communication uses the OpenAI API format, which has become the de facto standard for LLM interactions. Any new tool that speaks this format can slot into the stack immediately without modification.

â■ Composable by design: Need only browser automation? Run Browser Use alone. Need desktop + code? Combine Agent-S and OpenHands. The stack scales up and down based on your actual needs, without forcing you to install or maintain components you do not use.